Name: Gunjan Rupani
Roll No: 2401096
Batch: T22
Subject: Agile Software Development & DevOps

Experiment: 01

Aim: To perform version control operations using Git.

Learning Outcome:

- Understand Git and GitHub.
- Perform basic Git operations.
- Learn branching and merging.
- Handle version history and rollback operations.
- Understand Git workflow.

Software Required:

- Git
- Git Bash
- Visual Studio Code / Notepad

Theory:

Git is a distributed version control system that helps developers track changes made to source code. It allows multiple developers to work on the same project efficiently by maintaining different versions of files. Git provides features such as branching, merging, rollback, and history tracking, making software development organized and reliable.

Procedure:

Setup & Configuration

1. git --version — Checks which version of Git is installed on the system.

2. git config --global user.name "Your Name" — Sets the username attached to your commits.

3. git config --global user.email "your_email@gmail.com" — Sets the email attached to your commits.

Creating the Repository

4. mkdir git_experiment — Creates a new folder for the project.

5. cd git_experiment — Moves into the project folder.

6. git init — Initializes a new Git repository in the current folder.

7. git status — Displays the current status of the working directory, including tracked, untracked, and staged files.

Creating & Committing Files

8. echo 'print("Hello World - Version 1")' > app.py — Creates a Python file with the initial program.

9. git add app.py — Stages the Python file for committing.

10. git commit -m "Version 1: initial hello world" — Saves the staged changes as the first commit.

11. git log --oneline — Displays the commit history in a compact one-line format.

Editing & Comparing Versions

12. echo 'print("Hello World - Version 2")' > app.py — Modifies the Python file to create Version 2.

13. git diff — Shows the differences between the modified file and the last committed version.

14. git add . — Stages all modified and newly created files.

15. git commit -m "Version 2: updated message" — Commits the updated version of the project.

Branching

16. git branch — Lists all branches and highlights the currently active branch.

17. git branch feature-branch — Creates a new branch named **feature-branch**.

18. git checkout feature-branch — Switches to the **feature-branch**.

19. echo 'print("Hello World - Version 3 on feature branch")' > app.py — Updates the Python file in the feature branch.

20. git commit -am "Version 3: edited on feature branch" — Stages tracked changes and commits them in a single command.

21. git checkout master — Switches back to the master branch.

22. git merge feature-branch — Merges the changes from **feature-branch** into the current branch.

23. git checkout -b test-branch — Creates a new branch named **test-branch** and switches to it.

Undoing Changes / Rollback

24. git checkout master — Returns to the master branch.

25. echo temp change > app.py — Creates a temporary modification in the Python file.

26. git restore app.py — Discards the uncommitted changes and restores the file to its last committed state.

27. git reset --soft HEAD~1 — Undoes the last commit but keeps changes staged.

28. git reset --hard HEAD~1 — Completely undoes the last commit and discards the changes (destructive).

29. git revert <commit-id> — Creates a new commit that undoes an older commit's changes, without deleting history (the safe rollback method).

30. git log --oneline — Displays the final commit history after completing all Git operations.

Result:

1.git --version

```
PS C:\Users\gunja> git --version
git version 2.55.0.windows.3
PS C:\Users\gunja>
```

2.git config --global user.name "Your Name"

```
PS C:\Users\gunja> git config --global user.name "Gunjan Rupani"
```

3.git config --global user.email "your_email@gmail.com"

```
PS C:\Users\gunja> git config --global user.email "gunjan.rupaniii@gmail.com"
```

4.mkdir git_experiment

```
PS C:\Users\gunja> mkdir git_experiment


    Directory: C:\Users\gunja


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        26-07-2026     15:57                git_experiment


PS C:\Users\gunja>
```

5.cd git_experiment

```
PS C:\Users\gunja> cd git_experiment
```

6.git init

```
PS C:\Users\gunja\git_experiment> git init
Initialized empty Git repository in C:/Users/gunja/git_experiment/.git/
PS C:\Users\gunja\git_experiment>
```

7.git status

```
PS C:\Users\gunja\git_experiment> git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
PS C:\Users\gunja\git_experiment>
```

8.echo 'print("Hello World - Version 1")' > app.py

```
PS C:\Users\gunja\git_experiment> echo 'print("Hello World - Version 1")' > app.py
PS C:\Users\gunja\git_experiment> cat app.py
print("Hello World - Version 1")
PS C:\Users\gunja\git_experiment>
```

9.git add app.py

```
PS C:\Users\gunja\git_experiment> git add app.py
```

10.git commit -m "Version 1: initial hello world"

```
PS C:\Users\gunja\git_experiment> git commit -m "Version 1: initial hello world"
[main (root-commit) 6aa7de7] Version 1: initial hello world
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 app.py
PS C:\Users\gunja\git_experiment>
```

11.git log --oneline

```
PS C:\Users\gunja\git_experiment> git log --oneline
6aa7de7 (HEAD -> main) Version 1: initial hello world
PS C:\Users\gunja\git_experiment>
```

12.echo 'print("Hello World - Version 2")' > app.py

```
PS C:\Users\gunja\git_experiment> echo 'print("Hello World - Version 2")' > app.py
```

13.git diff

```
PS C:\Users\gunja\git_experiment> git diff
diff --git a/app.py b/app.py
index dabba07..0d7be4c 100644
Binary files a/app.py and b/app.py differ
PS C:\Users\gunja\git_experiment>
```

14.git add .

```
PS C:\Users\gunja\git_experiment> git add .
```

15.git commit -m "Version 2: updated message"

```
PS C:\Users\gunja\git_experiment> git commit -m "Version 2: updated message"
[main 09ec592] Version 2: updated message
 1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\gunja\git_experiment>
```

16.git branch

```
PS C:\Users\gunja\git_experiment> git branch
* main
PS C:\Users\gunja\git_experiment>
```

17.git branch feature-branch

```
PS C:\Users\gunja\git_experiment> git branch feature-branch
```

18.git checkout feature-branch

```
PS C:\Users\gunja\git_experiment> git checkout feature-branch
Switched to branch 'feature-branch'
PS C:\Users\gunja\git_experiment>
```

19.echo 'print("Hello World - Version 3 on feature branch")' > app.py

```
PS C:\Users\gunja\git_experiment> echo 'print("Hello World - Version 3 on feature branch")' > app.py
PS C:\Users\gunja\git_experiment>
```

20.git commit -am "Version 3: edited on feature branch"

```
PS C:\Users\gunja\git_experiment> git commit -am "Version 3: edited on feature branch"
[feature-branch f5c3e3d] Version 3: edited on feature branch
 1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\gunja\git_experiment>
```

21.git checkout main

```
PS C:\Users\gunja\git_experiment> git checkout main
Switched to branch 'main'
PS C:\Users\gunja\git_experiment>
```

22.git merge feature-branch

```
PS C:\Users\gunja\git_experiment> git merge feature-branch
Updating 09ec592..f5c3e3d
Fast-forward
 app.py | Bin 70 -> 106 bytes
 1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\gunja\git_experiment>
```

23.git checkout -b test-branch

```
PS C:\Users\gunja\git_experiment> git checkout -b test-branch
Switched to a new branch 'test-branch'
```

24.git checkout master

```
PS C:\Users\gunja\git_experiment> git checkout main
Already on 'main'
PS C:\Users\gunja\git_experiment>
```

25.echo temp change > app.py

```
PS C:\Users\gunja\git_experiment> echo temp change > app.py
PS C:\Users\gunja\git_experiment>
```

26.git restore app.py

```
PS C:\Users\gunja\git_experiment> git restore app.py
```

27.echo another temp change > app.py

git add app.py

git restore --staged app.py

```
PS C:\Users\gunja\git_experiment> echo temp change > app.py
PS C:\Users\gunja\git_experiment> git restore app.py
PS C:\Users\gunja\git_experiment> echo another temp change > app.py
PS C:\Users\gunja\git_experiment> git add app.py
PS C:\Users\gunja\git_experiment> git restore --staged app.py
PS C:\Users\gunja\git_experiment> git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.py

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\gunja\git_experiment>
```

28.git add app.py

git commit -m "temp commit for reset demo"

git reset --soft HEAD~1

git status

```
PS C:\Users\gunja\git_experiment> git add app.py
PS C:\Users\gunja\git_experiment> git commit -m "temp commit for reset demo"
[main 75b4c70] temp commit for reset demo
 1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\gunja\git_experiment> git reset --soft HEAD~1
PS C:\Users\gunja\git_experiment> git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   app.py
```

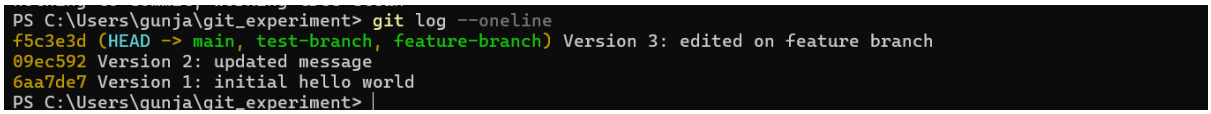
29.git commit -m "temp commit for hard reset demo"

git reset --hard HEAD~1

git status

```
PS C:\Users\gunja\git_experiment> git commit -m "temp commit for hard reset demo"
[main 1e18716] temp commit for hard reset demo
 1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\gunja\git_experiment> git reset --hard HEAD~1
HEAD is now at f5c3e3d Version 3: edited on feature branch
PS C:\Users\gunja\git_experiment> git status
On branch main
nothing to commit, working tree clean
PS C:\Users\gunja\git_experiment>
```

30.git log --oneline



Conclusion

This experiment provided practical experience with Git version control, covering repository initialization, committing changes, branching, merging, and rolling back versions using restore and reset commands. It demonstrated how Git maintains project history and enables developers to safely manage and recover different versions of their code.